

JavaScript_Day-4_Hands_On—Vasundhara

Problem 1: Personal Notes Saver using LocalStorage (Level-1)

Scenario

You are building a simple web page where users can write daily notes and save them in their browser without using a server.

Requirements

- ☐ A textarea for writing notes.
- ☐ A **Save** button (using onclick).
- ☐ A **Clear** button.
- ☐ Notes must:

- Be stored in localStorage

- Automatically load when the page refreshes

- ☐ Display stored note on page load.

Technical Constraints

- ☐ Must use:
 - onclick inline event
 - localStorage.setItem()
 - localStorage.getItem()
 - localStorage.removeItem()
- ☐ No backend/database.
- ☐ Pure HTML + JavaScript only.
- ☐ Data stored as key-value pair.

🎯 Learning Outcome

You should be able to:

- ❑ Inline event handling (onclick)
- ❑ Browser Storage APIs
- ❑ Storing & retrieving key-value data

Page load event handling

Source code:

```
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Personal Notes </title>
8      <style>
9          body { font-family: Arial, sans-serif; margin: 20px; }
10         textarea { width: 100%; max-width: 600px; height: 200px; padding: 10px; }
11         button { margin: 10px 5px 0 0; padding: 10px 20px; font-size: 16px; }
12     </style>
13 </head>
14 <body>
15
16     <h2>My Daily Note</h2>
17
18     <textarea id="note" placeholder="Write your notes here..."></textarea>
19     <br>
20
21     <button onclick="saveNote()">Save</button>
22     <button onclick="clearNote()">Clear</button>
23
24     <script>
25         // Load saved note when page opens
26         window.onload = function() {
27             let savedNote = localStorage.getItem("myDailyNote");
28             if (savedNote) {
29                 document.getElementById("note").value = savedNote;
30             }
31         };
32
33         // Save function
34         function saveNote() {
35             let text = document.getElementById("note").value;
36             localStorage.setItem("myDailyNote", text);
37             alert("Note saved successfully!"); // optional feedback
38         }
39
40         // Clear function
41         function clearNote() {
42             document.getElementById("note").value = "";
43             localStorage.removeItem("myDailyNote");
44             alert("Note cleared!"); // optional
45         }
46     </script>
47
48 </body>
49 </html>
50
```

Output:

My Daily Note

hi

Save

Clear

/

127.0.0.1:5500 says

Note saved successfully!

OK

My Daily Note

hi

Save

Clear

/

Key	Value
locations	[{"latitude":17.4771,"longitude...]
myDailyNote	hi

Problem 2: Live Form Validation with Events (Level-1)

Scenario

Create a simple registration form that validates user input when fields change.

Requirements

- ☐ Fields:

Name

Email

Age

☐ Use:

onchange

onclick

☐ Validate:

Name cannot be empty

Email must contain "@"

Age must be greater than 18

☐ Display validation message dynamically.

☐ Store valid user data in sessionStorage.

Technical Constraints

☐ Must use inline onchange events.

☐ Store valid data using:

sessionStorage.setItem()

☐ No external libraries.

☐ Use basic JavaScript only.

Learning Outcome

You will be able to:

☐ onchange event usage

☐ Form validation logic

☐ Difference between localStorage and sessionStorage

☐ Dynamic DOM updates

Source code:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Live Form Validation</title>
5      <style>
6          .error {
7              color: red;
8          }
9          .success {
10             color: green;
11         }
12     </style>
13 </head>
14 <body>
15
16     <h2>Registration Form</h2>
17
18     <label>Name:</label><br>
19     <input type="text" id="name" onchange="validateName()">
20     <p id="nameMsg"></p>
21
22     <label>Email:</label><br>
23     <input type="text" id="email" onchange="validateEmail()">
24     <p id="emailMsg"></p>
25
26     <label>Age:</label><br>
27     <input type="number" id="age" onchange="validateAge()">
28     <p id="ageMsg"></p>
```

```
30 <br>
31 <button onclick="submitForm()">Submit</button>
32
33 <p id="finalMsg"></p>
34
35 <script>
36
37 function validateName() {
38     var name = document.getElementById("name").value;
39     var msg = document.getElementById("nameMsg");
40
41     if (name === "") {
42         msg.innerText = "Please enter name";
43         msg.className = "error";
44         return false;
45     } else {
46         msg.innerText = "Valid name";
47         msg.className = "success";
48         return true;
49     }
50 }
51
52 function validateEmail() {
53     var email = document.getElementById("email").value;
54     var msg = document.getElementById("emailMsg");
55
56     if (email === "") {
```

```
50 }
51
52 function validateEmail() {
53     var email = document.getElementById("email").value;
54     var msg = document.getElementById("emailMsg");
55
56     if (email === "") {
57         msg.innerText = "Please enter email";
58         msg.className = "error";
59         return false;
60     } else if (email.indexOf("@") === -1) {
61         msg.innerText = "Invalid email";
62         msg.className = "error";
63         return false;
64     } else {
65         msg.innerText = "Valid email";
66         msg.className = "success";
67         return true;
68     }
69 }
70
71 function validateAge() {
72     var age = document.getElementById("age").value;
73     var msg = document.getElementById("ageMsg");
74 }
```

```

74
75     if (age === "") {
76         msg.innerText = "Please enter age";
77         msg.className = "error";
78         return false;
79     } else if (age <= 18) {
80         msg.innerText = "Age must be greater than 18";
81         msg.className = "error";
82         return false;
83     } else {
84         msg.innerText = "Valid age";
85         msg.className = "success";
86         return true;
87     }
88 }
89
90 function submitForm() {
91     var isNameValid = validateName();
92     var isEmailValid = validateEmail();
93     var isAgeValid = validateAge();
94
95     var finalMsg = document.getElementById("finalMsg");
96
97     if (isNameValid && isEmailValid && isAgeValid) {
98         sessionStorage.setItem("name", name.value);
99
100     }
101
102     finalMsg.innerText = "Data stored successfully";
103     finalMsg.className = "success";
104 } else {
105     finalMsg.innerText = "Please fill all fields correctly";
106     finalMsg.className = "error";
107 }
108 }
109
110 </script>
111
112 </body>
113 </html>

```

Output:

Registration Form

Name:

Valid name

Email:

Valid email

Age:

Valid age

Data stored successfully

Problem 3: Location-Based Weather Logger (Level-2)

Scenario

Create a web application that fetches the user's geographic location and stores location history in localStorage.

Requirements

- ☐ Button: **Get My Location**
- ☐ Use:
 navigator.geolocation.getCurrentPosition()
- ☐ Display:
 Latitude
 Longitude
- ☐ Handle:
 Permission denied
 Timeout
 Location unavailable
- ☐ Save last 5 location entries in localStorage.
- ☐ Display location history on page load.

Technical Constraints

- ☐ Must handle:
 Success callback
 Error callback
- ☐ Use browser permission handling.
- ☐ Store data as JSON using:
 JSON.stringify()
 JSON.parse()
- ☐ Use inline event (onclick).

Learning Outcome

Learners should be able to:

- Geolocation API
- Handling browser permissions
- Error handling in APIs
- Managing structured data in localStorage
- JSON parsing and stringifying

Source code:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Location-Based Weather Logger</title>
5      <style>
6          body {
7              font-family: Arial, sans-serif;
8          }
9          .error {
10             color: red;
11         }
12         .success {
13             color: green;
14         }
15     </style>
16 </head>
17
18 <body onload="loadHistory()">
19
20 <h2>Location-Based Weather Logger</h2>
21
22 <button onclick="getLocation()">Get My Location</button>
23
24 <p id="status"></p>
25
26 <h3>Current Location</h3>
27 <p id="lat"></p>
28 <p id="lon"></p>
```

```
28 <p id="lon"></p>
29
30 <h3>Location History (Last 5)</h3>
31 <ul id="history"></ul>
32
33 <script>
34
35 function getLocation() {
36     var status = document.getElementById("status");
37
38     if (navigator.geolocation) {
39         navigator.geolocation.getCurrentPosition(
40             showPosition,
41             showError
42         );
43     } else {
44         status.innerHTML = "Geolocation not supported";
45         status.className = "error";
46     }
47 }
48
49 /* SUCCESS CALLBACK */
50 function showPosition(position) {
51     var latitude = position.coords.latitude;
52     var longitude = position.coords.longitude;
53
54     document.getElementById("lat").innerHTML = "Latitude: " + latitude;
```



```

50  function showPosition(position) {
55      document.getElementById("lon").innerText = "Longitude: " + longitude;
56
57      document.getElementById("status").innerText = "Location fetched successfully";
58      document.getElementById("status").className = "success";
59
60      saveLocation(latitude, longitude);
61  }
62
63  /* ERROR CALLBACK */
64  function showError(error) {
65      var status = document.getElementById("status");
66      status.className = "error";
67
68      if (error.code === 1) {
69          status.innerText = "Permission denied";
70      } else if (error.code === 2) {
71          status.innerText = "Location unavailable";
72      } else if (error.code === 3) {
73          status.innerText = "Request timeout";
74      }
75  }
76
77  /* SAVE LAST 5 LOCATIONS */
78  function saveLocation(lat, lon) {
79      var locations = JSON.parse(localStorage.getItem("locations")) || [];

```

```

103      var locations = JSON.parse(localStorage.getItem("locations")) || [];
104
105      locations.forEach(function(item) {
106          var li = document.createElement("li");
107          li.innerText =
108              "Lat: " + item.latitude +
109              ", Lon: " + item.longitude +
110              " (" + item.time + ")";
111          historyList.appendChild(li);
112      });
113  }
114
115  </script>
116
117  </body>
118  </html>

```

Output:

Location-Based Weather Logger

Get My Location

Location fetched successfully

Current Location

Latitude: 17.4771

Longitude: 78.5724

Location History (Last 5)

- Lat: 17.4771, Lon: 78.5724 (2/23/2026, 10:38:09 PM)
- Lat: 17.4771, Lon: 78.5724 (2/23/2026, 10:38:09 PM)
- Lat: 17.4771, Lon: 78.5724 (2/23/2026, 10:37:22 PM)
- Lat: 17.4771, Lon: 78.5724 (2/23/2026, 10:37:22 PM)
- Lat: 17.4771, Lon: 78.5724 (2/23/2026, 10:37:22 PM)

Problem 4: Mini Expense Tracker using Client-Side Database (Level-2)

Scenario

Develop a client-side expense tracker where users can add, view, and delete expenses using a browser-supported database.

Requirements

1. Fields:

- Expense Title
- Amount
- Date

2. Buttons:

- Add Expense
- View Expenses
- Delete Expense

3. Use:

- Client-side database (Web SQL or IndexedDB)

4. Execute SQL-like queries:
 - CREATE TABLE
 - INSERT
 - SELECT
 - DELETE
5. Maintain transaction handling.
6. Display expense list dynamically.

Technical Constraints

- ☐ Must use:
 - Client-side DB transactions
 - SQL execution methods
- ☐ Must handle:
 - Transaction errors
 - Query errors
- ☐ No server/database allowed.
- ☐ Pure JavaScript + HTML.

Learning Outcome

You will be able to:

- ☐ Client-side database concepts
- ☐ Executing SQL queries in browser
- ☐ Managing transactions
- ☐ Advanced DOM rendering
- ☐ Persistent structured storage

Source code:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Mini Expense Tracker</title>
5  </head>
6  <body>
7
8  <h2>Mini Expense Tracker</h2>
9
10 <label>Expense Title:</label><br>
11 <input type="text" id="title"><br>
12
13 <label>Amount:</label><br>
14 <input type="number" id="amount"><br>
15
16 <label>Date:</label><br>
17 <input type="date" id="date"><br><br>
18
19 <button onclick="addExpense()">Add Expense</button>
20 <button onclick="viewExpenses()">View Expenses</button>
21
22 <h3>Expense List</h3>
23 <ul id="expenseList"></ul>
24
25 <script>
26
27 var db;
28 var request = indexedDB.open("ExpenseDB", 1);
29
```

```
25 <script>
26
28 var request = indexedDB.open("ExpenseDB", 1);
29
30 request.onupgradeneeded = function(event) {
31 |   db = event.target.result;
32 |   db.createObjectStore("expenses", { keyPath: "id", autoIncrement: true });
33 | };
34
35 request.onsuccess = function(event) {
36 |   db = event.target.result;
37 | };
38
39 function addExpense() {
40 |   var title = document.getElementById("title").value;
41 |   var amount = document.getElementById("amount").value;
42 |   var date = document.getElementById("date").value;
43
44 |   if (title === "" || amount === "" || date === "") {
45 |     alert("Please fill all fields");
46 |     return;
47 |   }
48
49 |   var transaction = db.transaction(["expenses"], "readwrite");
50 |   var store = transaction.objectStore("expenses");
51
52 |   store.add({
53 |     title: title,
```

```

39  function addExpense() {
52      store.add({
53          title: title,
54          amount: amount,
55          date: date
56      });
57
58      transaction.oncomplete = function() {
59          alert("Expense Added Successfully");
60          viewExpenses();
61      };
62  }
63
64  function viewExpenses() {
65      var list = document.getElementById("expenseList");
66      list.innerHTML = "";
67
68      var transaction = db.transaction(["expenses"], "readonly");
69      var store = transaction.objectStore("expenses");
70
71      store.openCursor().onsuccess = function(event) {
72          var cursor = event.target.result;
73          if (cursor) {
74              var li = document.createElement("li");
75              li.innerText =
76                  cursor.value.title + " - ₹" +
64  function viewExpenses() {
65      var list = document.getElementById("expenseList");
66      list.innerHTML = "";
67
68      var transaction = db.transaction(["expenses"], "read
69      var store = transaction.objectStore("expenses");
70
71      store.openCursor().onsuccess = function(event) {
72          var cursor = event.target.result;
73          if (cursor) {
74              var li = document.createElement("li");
75              li.innerText =
76                  cursor.value.title + " - ₹" +
77                  cursor.value.amount + " (" +
78                  cursor.value.date + ")";
79              list.appendChild(li);
80              cursor.continue();
81          }
82      };
83  }
84
85  </script>
86
87  </body>
88  </html>

```

Output:

Mini Expense Tracker

Expense Title:

Amount:

Date:

Expense List

127.0.0.1:5500 says
Expense Added Successfully

Mini Expense Tracker

Expense Title:

Tea

Amount:

22

Date:

02 / 04 / 2026



Add Expense

View Expenses

Expense List

- Tea - ₹24 (2026-02-02)
- Tea - ₹22 (2026-02-04)

